SCAMSHIELD AI

India's Ultimate Cyber Fraud Detection & Incident Response Platform

COMPLETE TECHNICAL PRODUCT DOCUMENTATION

Developer: Nitin Kushwah, IIT Guwahati
Version: 1.0.0 (Production-Grade Release)
Date: June 2026

1. Executive Summary & Vision

In 2023 alone, citizens in India suffered accumulated losses exceeding Rs. 1.6 Lakh Crore to online and cybercrime fraud. The vast majority of scam victims face panic due to three primary gaps: (1) Uncertainty around whether a communication is indeed a scam, (2) Lack of knowledge of standard incident responses, and (3) Inability to easily construct formal complaint requests for banks and law enforcement.

ScamShield AI addresses this crisis by offering an automated, centralized cyber fraud response service. A victim simply copies and pastes a suspicious forward, message, call transcript, or email. The ScamShield AI analysis engine processes the content, yields an instant risk verdict (Low, Medium, High, Critical), explains the scam mechanics, identifies specific red flags, maps out numbered recovery steps, notes the legal sections (IPC/IT Act) that apply, and provides a portal to generate complaint documents. ScamShield AI is not a simple spam filter—it is a comprehensive cyber incident response assistant.

2. System Architecture

The product employs a robust two-tier client-server layout designed for horizontal scaling, rapid response cycles, and complete offline survivability:

Backend Application Tier (Node.js & Express):

Responsible for secure JWT sessions, rate limiting, request validation, PostgreSQL database queries, and AI provider scheduling. It incorporates express-rate-limit to protect auth routes (max 20 requests per 15 min) and API routes (max 200 requests per 15 min).

Frontend Client Tier (Next.js 14 & Tailwind CSS):

A state-of-the-art Next.js App Router workspace utilizing React Hook Form + Zod validation, responsive custom design tokens, dark/light theme adjustments, Lucide React icons, and Markdown-rendered AI responses.

Primary & Fallback AI Infrastructure:

The AI Analyzer operates with a multi-provider fallback loop: Google Gemini 1.5 Flash is checked first; if it returns rate limits (HTTP 429) or failures, the engine rotates credentials and calls Groq (Llama-3.3-70b). In the event that all external APIs are unreachable, a local rules-based regex keyword engine (scamPrompts.js) evaluates the text and structures a standard classification to ensure 100% service uptime.

3. Database Relational Models

ScamShield uses a clean, relational PostgreSQL schema to represent case histories, evidence tracking check sheets, community-based number bans, and documents:

Users Table Schema:

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  city VARCHAR(100),
  state VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Cases Table Schema (AI Analysis Logs):

```
CREATE TABLE cases (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  original_message TEXT NOT NULL,
  scam_type VARCHAR(200),
  scam_category VARCHAR(100),
  severity VARCHAR(20),
  confidence_score INTEGER,
  is_scam BOOLEAN,
  how_it_works TEXT,
  red_flags TEXT[],
  action_steps TEXT[],
  relevant_law TEXT,
  status VARCHAR(50) DEFAULT 'reported',
  created_at TIMESTAMP DEFAULT NOW()
);
```

Evidence Items Table Schema:

```
CREATE TABLE evidence_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  case_id UUID REFERENCES cases(id) ON DELETE CASCADE,
  item_name VARCHAR(200) NOT NULL,
  is_collected BOOLEAN DEFAULT false,
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

4. Core AI Analysis Prompt Design

The AI engine uses highly structured prompt engineering to command models to return JSON output without markdown delimiters. This ensures direct programmatic ingestion. The schema matches:

```
{
  "is_scam": true || false,
  "scam_type": "Specific scam classification",
  "scam_category": "financial_fraud" | "job_employment" | "prize_lottery" | ...,
  "severity": "Low" | "Medium" | "High" | "Critical",
  "confidence_score": 87,
  "confidence_label": "High - 87% likely scam",
  "how_it_works": "3-5 sentences detailed explanation of scam model",
  "red_flags": ["visual cues", "requests made", "irregularities"],
  "action_steps": ["step 1", "step 2"],
  "relevant_law": "IPC / IT Act sections",
  "evidence_to_collect": ["file items to gather"]
}
```

The local rule fallback parses keywords (e.g. UPI, AnyDesk, KBC, Narcotics) to yield the exact same JSON format structure. This dual approach ensures high-quality intelligence and zero crashes.

5. Installation & Deployment Guide

To build and boot ScamShield AI locally, follow the steps below in order:

Step 1: Database Migration

Ensure PostgreSQL is active. Run migrations to setup tables:

```
cd backend
npm install
node scripts/createTables.js
```

Step 2: Seed Community Database

Seed the community reported numbers database with initial demo entries:

```
node scripts/seedCommunity.js
```

Step 3: Run the Express Server

Start development server. It will listen on PORT 5000:

```
npm run dev
```

Step 4: Launch Frontend

In a separate terminal, install frontend dependencies and run Next.js:

```
cd ../frontend
npm install
npm run dev
```